UNISALES

RELATÓRIO DE ATIVIDADE

Revisar material disponibilizado pelo professor e adaptar

Projeto AdventureWorks Data Warehouse

Disciplina: Projeto de Análise e Fluxo de Dados - OLAP e ETL

Status da atividade: CONCLUÍDA

26 de outubro de 2026

1. Objetivo da atividade

A atividade teve como objetivo revisar o material disponibilizado pelo professor para execução de ETL com Apache Airflow, Docker e PostgreSQL, identificar quais elementos poderiam ser reutilizados no projeto individual AdventureWorks Data Warehouse e adaptar essa base à arquitetura, ao modelo dimensional e aos requisitos específicos do trabalho. A proposta não foi copiar o exemplo didático, mas reaproveitar sua estrutura de execução e integrá-la ao código ETL previamente desenvolvido e ao Star Schema consolidado para o AdventureWorks2016.

Como critério de conclusão, além da revisão conceitual do material, foram exigidas adaptações funcionais e testes práticos capazes de comprovar as conexões, as cargas no Data Warehouse, a idempotência das operações e o comportamento incremental baseado em watermark.

2. Material disponibilizado pelo professor revisado

A base de referência analisada foi o conteúdo da pasta aula.etl.2 do repositório da disciplina. O material apresenta uma infraestrutura Docker para Apache Airflow e PostgreSQL, juntamente com exemplos de DAGs e operadores SQL voltados a fins didáticos.

Elemento revisado	Uso identificado no projeto
Apache Airflow 3.3.1	Base de orquestração das futuras DAGs do projeto.
PostgreSQL 16	Banco utilizado pelo Airflow e referência para o serviço PostgreSQL do DW.
LocalExecutor	Executor mantido para o ambiente local do projeto.
docker-compose.yml	Estrutura principal reaproveitada e posteriormente adaptada.
.env.example	Modelo de configuração por variáveis de ambiente.
etl_build_schema_dw.py	Referência para uso de SQLExecuteQueryOperator e execução de scripts SQL.
etl_exemplo.py	Referência para organização de DAGs e tarefas.
Demais exemplos Blockbuster/STG	Mantidos apenas como referência didática, sem reutilização de regras de negócio.

3. Decisões de reutilização

Após a revisão, os componentes foram classificados em três grupos: reutilizar, adaptar e utilizar apenas como referência. Essa separação evitou incorporar ao projeto regras de negócio ou estruturas de dados que pertenciam somente aos exemplos do professor.

Classificação	Componentes
Reutilizar	Airflow 3.3.1, PostgreSQL 16, LocalExecutor, organização geral do ambiente Docker.
Adaptar	docker-compose.yml, .env.example, padrão @dag/@task, montagem de volumes, serviços PostgreSQL e integração com os módulos ETL.
Referência	SQL e DAGs do cenário Blockbuster, staging didático e exemplos que não representam o AdventureWorks.

4. Adaptações realizadas no ambiente

4.1 Docker Compose e serviços PostgreSQL

O docker-compose.yml do professor foi transformado em uma infraestrutura específica do projeto AdventureWorks. O nome do projeto Docker foi alterado para adventureworks-etl para evitar colisões com o ambiente de aula. Também foi criado um segundo serviço PostgreSQL, denominado postgres-dw, dedicado exclusivamente ao Data Warehouse.

- PostgreSQL de metadados do Airflow mantido como serviço postgres, com porta externa ajustada para 5434.
- PostgreSQL do Data Warehouse criado como postgres-dw, com banco adventureworks_dw e porta externa 5433.
- Volumes persistentes separados para metadados do Airflow e dados do DW.
- Montagem das pastas etl/ e sql/ em /opt/airflow/etl e /opt/airflow/sql.
- PYTHONPATH definido como /opt/airflow para permitir importações dos módulos ETL nas futuras DAGs.
- Dependências de inicialização ajustadas para considerar a disponibilidade do PostgreSQL do DW.

4.2 Conexões e variáveis de ambiente

O módulo etl/conexoes.py foi adaptado para eliminar credenciais fixas no código. A conexão com o SQL Server passou a utilizar variáveis de ambiente e ODBC Driver 18 for SQL Server, enquanto a conexão com o PostgreSQL do DW passou a usar parâmetros DW_POSTGRES_*. O arquivo .env permanece fora do versionamento por meio do .gitignore, enquanto o .env.example deve conter somente placeholders.

```
SQL Server (origem): host.docker.internal:1433 / AdventureWorks2016
PostgreSQL (destino): postgres-dw:5432 / adventureworks_dw
```

5. Adaptação da ETL ao modelo dimensional

O código ETL reaproveitado de um projeto anterior foi reorganizado para refletir o modelo dimensional final. Foram removidas dependências de dimensões que não pertencem ao novo desenho e os módulos foram divididos em extract, transform, load, incremental, control e tests.

Camada	Principais módulos adaptados
Extract	extrair_dim_produto.py, extrair_dim_territorio.py, extrair_dim_tempo.py, extrair_fato_venda.py
Transform	transformar_dim_produto.py, transformar_dim_territorio.py, transformar_dim_tempo.py, transformar_fato_venda.py
Load	carregar_dim_produto.py, carregar_dim_territorio.py, carregar_dim_tempo.py, carregar_fato_venda.py
Control	watermark.py e buscar_chaves_dimensoes.py
Incremental	pipeline_incremental.py

5.1 Estratégia incremental

Foi criada a tabela controle.etl_watermark para armazenar a última data de alteração processada por dim_produto, dim_territorio e fato_venda. A dimensão tempo não utiliza watermark porque é gerada deterministicamente a partir do intervalo de datas da origem e usa ON CONFLICT DO NOTHING. Nos demais processos, a extração aplica ModifiedDate > watermark e o watermark somente é atualizado após uma carga bem-sucedida.

As cargas de produto e território utilizam SCD Tipo 1 por meio de ON CONFLICT ... DO UPDATE. A tabela fato utiliza a combinação (sales_order_id, sales_order_detail_id) como proteção do grão. Dessa forma, a reexecução do processo é idempotente e não cria duplicidades.

6. Validações de infraestrutura e conectividade

Validação	Resultado
Docker/Compose	Arquivo docker-compose.yml validado pelo comando docker compose config.
SQL Server a partir do container	Conexão concluída com sucesso; 504 produtos e 121.317 itens de SalesOrderDetail consultados.
PostgreSQL do DW	Serviço postgres-dw iniciado como healthy e conexão Python validada.
Modelo físico	Schemas dw e controle criados; quatro tabelas dimensionais/fato e tabela etl_watermark disponíveis.
Compilação Python	Módulos ETL compilados no container Airflow sem SyntaxError.

7. Testes das dimensões

7.1 Dimensão Produto

A extração retornou 504 produtos. A transformação manteve os 504 registros, sem IDs nulos ou duplicados. Durante a primeira tentativa de carga foi identificado um erro de compatibilidade: o valor de substituição "Não informado" excedia o limite VARCHAR(10) da coluna tamanho. A transformação foi corrigida para usar "N/A" nessa coluna. Após a correção, todos os limites textuais foram validados e a carga foi concluída.

Teste	Resultado
Extração	504 registros
Transformação	504 registros; 0 IDs nulos; 0 duplicados
Carga	504 produtos processados
Total no DW	505 registros, considerando o membro desconhecido -1
Idempotência	505 IDs distintos e 0 duplicados após nova execução

7.2 Dimensão Território

Foram extraídos e transformados 10 territórios. Os comprimentos de nome, código de país e grupo territorial foram comparados com os limites do modelo físico antes da carga. A tabela final possui 11 registros, incluindo o membro desconhecido, e a reexecução manteve 11 IDs distintos e nenhuma duplicidade.

7.3 Dimensão Tempo

A dimensão tempo foi gerada a partir do menor e maior OrderDate da origem. Foram produzidas 1.127 datas, entre 31/05/2011 e 30/06/2014, com chave no formato YYYYMMDD. Não foram encontradas SKs ou datas duplicadas. A segunda execução manteve exatamente 1.127 registros, demonstrando idempotência.

8. Testes da tabela fato

A extração da fato retornou 121.317 itens de venda, correspondentes a 31.465 pedidos. A combinação de SalesOrderID e SalesOrderDetailID apresentou zero duplicidades, confirmando o grão de um registro por item de pedido. A regra de custo foi validada com uso prioritário do histórico vigente na data da venda e fallback para o custo atual do produto.

Indicador de validação	Resultado
Itens extraídos/transformados	121.317
Pedidos distintos	31.465
Duplicidades no grão	0
SK produto desconhecida	0
SK território desconhecida	0
Custos/margens nulos	0
Origem de custo HISTORICO	121.253 itens
Origem de custo FALLBACK_ATUAL	64 itens
Receita total	109.846.381,40
Custo total	97.288.600,80
Margem bruta total	12.557.780,60

Após a carga, a tabela dw.fato_venda permaneceu com 121.317 registros e 121.317 grãos distintos mesmo depois de uma segunda execução, comprovando idempotência.

9. Validação do ETL incremental

9.1 Inicialização e reexecução sem alterações

Na primeira execução do pipeline_incremental.py, os watermarks ainda estavam nulos, portanto o pipeline reprocessou o conjunto completo. Como as cargas são idempotentes, nenhuma duplicidade foi criada. Os watermarks passaram a armazenar os maiores ModifiedDate observados na origem.

Processo	1ª execução incremental	2ª execução sem alterações
dim_produto	504 extraídos / 504 processados	0 / 0
dim_territorio	10 / 10	0 / 0
dim_tempo	1.127 / 1.127	1.127 / 1.127*
fato_venda	121.317 / 121.317	0 / 0

* A dimensão tempo é regenerada integralmente, mas a carga utiliza ON CONFLICT DO NOTHING; por isso os 1.127 representam tentativas de processamento, não novas inserções.

9.2 Teste incremental controlado - produto

Para demonstrar a seleção de apenas registros posteriores ao watermark, o controle de dim_produto foi temporariamente posicionado em 08/02/2014 10:01:36.827. Na origem, 503 produtos possuíam esse ModifiedDate e apenas 1 produto possuía timestamp posterior, 08/02/2014 10:03:55.510. A execução incremental extraiu e processou exatamente 1 registro, atualizou automaticamente o watermark para o valor mais recente e manteve 505 produtos distintos no DW.

9.3 Teste incremental controlado - fato

Na fato, o watermark foi temporariamente posicionado em 06/07/2014. A origem continha 96 itens com ModifiedDate em 07/07/2014. A execução de executar_fato_venda processou exatamente esses 96 registros e atualizou o watermark para 07/07/2014, mantendo 121.317 grãos distintos e zero duplicidades.

Teste incremental	Esperado	Obtido
Produto	1 registro posterior ao watermark	1 extraído / 1 processado
Fato venda	96 registros posteriores ao watermark	96 extraídos / 96 processados
Integridade após reprocessamento	Sem duplicidades	0 duplicados em ambos os casos

10. Problemas identificados e correções

Problema	Tratamento aplicado
Conflito da porta 5432 no host	Porta externa dos metadados do Airflow alterada para 5434; comunicação interna permanece postgres:5432.
VARCHAR(10) em dim_produto.tamanho	Valor padrão alterado de "Não informado" para "N/A".
Comandos psql com escape no PowerShell	Uso de here-string para enviar SQL pela entrada padrão do psql.
Aviso do pandas com conexão pyodbc	Classificado como warning não bloqueante; consultas retornaram resultados corretos. Pode ser eliminado futuramente com SQLAlchemy.
Métrica de dim_tempo	Identificado que "processados" representa tentativas, não inserts efetivos; melhoria registrada para etapa posterior.

11. Situação final da atividade

A revisão do material do professor foi concluída e sua estrutura foi adaptada com sucesso ao projeto AdventureWorks Data Warehouse. O ambiente Docker foi especializado para o projeto, a ETL modular foi integrada ao Star Schema, as conexões com SQL Server e PostgreSQL foram validadas e o processo incremental foi comprovado em cenários de carga completa, reexecução sem alterações e reprocessamento de subconjuntos posteriores ao watermark.

Componente	Situação
Revisão do material do professor	Concluída
Classificação reutilizar/adaptar/referência	Concluída
Docker Compose adaptado	Concluído
Conexões SQL Server/PostgreSQL	Validadas
ETL das três dimensões	Validada
ETL da fato venda	Validada
Idempotência	Validada
Watermark e carga incremental	Validados
Orquestração em DAG do Airflow	Próxima etapa

12. Conclusão

A atividade atingiu seu objetivo: a base didática disponibilizada pelo professor foi compreendida, selecionada e adaptada de forma compatível com o escopo individual do projeto. Os testes realizados demonstraram que as adaptações não se limitaram à organização do código; elas foram executadas contra o AdventureWorks2016 e o PostgreSQL do DW, com validação quantitativa dos dados e do comportamento incremental. Dessa forma, a infraestrutura e a ETL estão preparadas para a próxima etapa, que consiste em orquestrar as rotinas por meio de uma DAG do Apache Airflow e registrar as evidências de execução na interface da ferramenta.